**Abstract**
Malicious URLs, often known as malicious websites, pose a prevalent and major danger to cybersecurity. Malicious URLs can lead to scams such as spam, phishing, and drive-by downloads, resulting in billions of dollars in damages annually. Such risks must be detected and addressed promptly. Blacklists have traditionally been used to detect this. However, blacklists are not complete and cannot detect new dangerous URLs. In recent years, there has been a growing interest in utilizing machine learning approaches to enhance the accuracy of harmful URL detection. This paper seeks to provide a detailed assessment and understanding of machine learning algorithms for detecting malicious URLs. We formalize Malicious URL Detection as a machine learning challenge and analyze literature on several aspects of the topic, including feature representation and algorithm design. This article offers a comprehensive survey for machine learning researchers and engineers in academia, as well as professionals and practitioners in the cybersecurity industry. It helps them understand the current state of the art and facilitates their research and practical applications. We address practical concerns in system design, and suggest future paths.

**Keywords:** Cybersecurity, Malicious URLs, Machine Learning, LightGBM, Feature Engineering, Classification.

**Introduction**
Cyber attacks have become a significant security concern as the Internet has grown in popularity. Malicious URLs are commonly used in various assaults, including phishing, Trojan horses, and malware. As the URL creation algorithm matures, dangerous URLs continue to emerge on a daily basis. Identifying bad URLs is crucial for preventing network attacks and ensuring security. The blacklist method is the most common way to detect dangerous URLs **[1]**.

Even if today's security components attempt to detect such malicious sites and web addresses, they evade detection by employing various tactics devised by attackers. Researchers studied to find effective techniques for detecting malicious URLs. One of the most frequent methods is to employ a blacklist of known dangerous URLs to screen incoming URLs. However, blacklists have restrictions, making this strategy ineffective for newly developed rogue sites. The Uniform Resource Locator (URL) is a universal address for documents and resources on the World Wide Web. A URL comprises two major components: (i) Protocol identifier (indicating the protocol to use). (ii) Resource name: gives the IP address or domain name of the resource.
Malicious URLs are those that have been compromised and are being used for cyber assaults. Approximately one-third of all websites are potentially harmful, indicating the widespread usage of bad URLs to commit cybercrimes. Malicious URLs and web sites use spam, phishing, and drive-by downloads to conduct assaults. Unsuspecting users may fall victim to frauds such as monetary loss, identity theft, and virus installation. Malicious URLs are commonly used for attacks such as drive-by downloads, phishing, social engineering, and spam **[2]**.

This chapter seeks to introduce the fundamentals of machine learning-based harmful URL detection. The remainder of the chapter is arranged as follows. The Background section provides a review of the existing approach and a synopsis of the literature in the topic of URL classification. In the Dataset and Analysis section, the publicly available dataset is discussed. The principles of machine-learning models are discussed in the Methods section. The Experiment section compares the detection performance of several algorithms. Finally, remarks are made about the conclusion and future directions.

## Related Work

Schultz et al. **[3]** used static analysis techniques to detect malware using machine learning. They focused on features such as extracted strings, header information from executable files (called and imported functions), and a contiguous sequence of n bytes (grams). The authors of the research compared three machine learning methods (RIPPER, naïve Bayes (NB), and multi-NB) to a standard malware detection method (signature-based detection). Using machine learning techniques resulted in significantly higher detection accuracy (83.62% to 97.11%) than traditional signature-based detection (49.28%).

Bai et al**. [4]** discussed the usage of static analysis approaches. The researchers analyzed 197 features from executable file headers, including API calls and dynamically linked libraries. The authors employed filtering and wrapping methods to identify the optimal attributes for distinguishing between malicious and benign samples. This study compared decision-tree-based algorithms, specifically J48 and random forest (RF). Several trials yielded detection accuracy ranging from 94.6% to 99.1%.

The authors of the aforementioned publications retrieved features utilizing static analysis approaches. Moser et al. **[5]** identified the main concern with this malware detection approach as the use of obfuscation techniques to disguise harmful code. They advocate adopting dynamic analysis, which is less prone to this issue.

Mosli et al. **[6]** employed Cuckoo Sandbox, an automated malware analysis system, to extract features from memory dumps generated during sample run. They evaluated the accuracy of three algorithms: KNN, SVM, and RF. They achieved the highest detection accuracy with the RF technique, at 91.4%. The KNN algorithm yielded the lowest detection accuracy (91%).

## Purposed Model

Data collection includes a tagged dataset of URLs classified as dangerous, benign, defacement, or malware, obtained from the Kaggle repository. The dataset does not include any null or empty cells. Pre-processing involves removing missing data, extracting new attributes, normalizing, encoding category values, and standardizing values.
The Sklearn Python library is used to train the model using various machine learning approaches, including random forest classifiers, Light GBM classifiers, and XGboost classifiers. Approximately 80% of the data is employed. Validate and optimize models using the remaining 20% of data. Hyperparameters are adjusted to improve sensitivity, F1 score, recall, and accuracy. Model comparison: Assessment measures are used to compare machine learning classification systems.

## Designing and Implementation

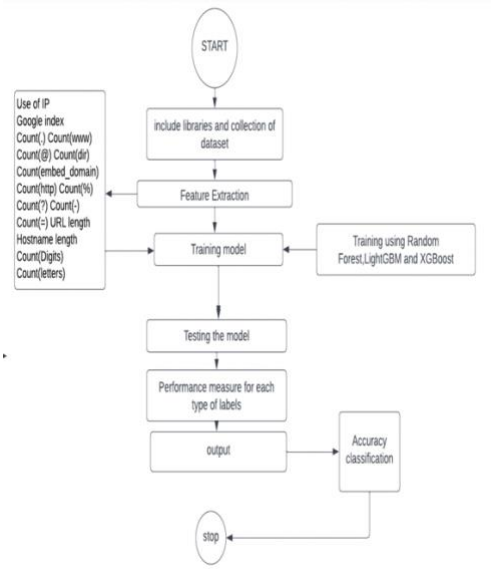



**Fig. 1.** Flowchart of design and Implimentation

Figure 1 depicts the design and implementation process, beginning with the

inclusion of necessary libraries and datasets. The dataset extracted features to improve model performance and predict output more accurately. Feature extraction involves reducing dimensionality to acquire only the necessary columns for predicting harmful URLs, rather than the entire dataset **[7].**

After obtaining the extracted features, the model must be trained with the specified classifiers. The model should be evaluated on an 80:20 split dataset, measured using multiple metrics, and compared to other models to determine the best fit.

### .1 Classification Techinques

Classification is a supervised machine learning method that involves feeding a dataset into a model with labels. The model will receive prior information about its training data. Data can be either organized or unstructured. The procedure includes preprocessing, model training, and data classification. Classes are sometimes known as labels, aims, or categories. The classes cover the entire dataset, while the labels apply to each individual dataset. Binomial classification and multi-class classification are two types of classification. Classification include categorizing emails as spam or not, recognizing positive or negative sentiment in tweets, and classifying sights such as fruits, animals, and insects.

The supervised learning method uses the well-known random forest ML algorithm. It can be used for regression and classification challenges. Figure 2 shows the code snippet and usage of the random forest model.

```
[27]: from sklearn.ensemble import RandomForestClassifier
      from sklearn import metrics
      rf = RandomForestClassifier(n_estimators=100,max_features='sqrt')
      rf.fit(X_train,y_train)
      y_pred_rf = rf.predict(X_test)
      print(classification_report(y_test,y_pred_rf,target_names=['benign', 'defacement','phis

      score = metrics.accuracy_score(y_test, y_pred_rf)
      print("accuracy:   %0.3f" % score)
```

**Fig. 2.** Random Foreset Classifier

The Light GBM classifier is a decision tree-based gradient boosting technique that optimizes model performance and memory usage. Figure 3 displays the code snippet for a light GBM model.

```
[40]: lgb = LGBMClassifier(objective='multiclass',boosting_type= 'gbdt',n_jobs = 5,
              silent = True, random_state=5)
      LGB_C = lgb.fit(X_train, y_train)



      y_pred_lgb = LGB_C.predict(X_test)
      print(classification_report(y_test,y_pred_lgb,target_names=['benign', 'defacement','phishing','malware']))

      score = metrics.accuracy_score(y_test, y_pred_lgb)
      print("accuracy:   %0.3f" % score)
```

Fig. 3. Light GBM Classifier

University of Washington researchers created the Extreme Gradient Boosting (XGboost) classifier. This C++ library improves training for gradient boosting. Figure 4 shows the code snippet for the XGboost classifier model.

```
[48]: xgb_c = xgb.XGBClassifier(n_estimators= 100)
      xgb_c.fit(X_train,y_train)
      y_pred_x = xgb_c.predict(X_test)
      print(classification_report(y_test,y_pred_x,target_names=['benign', 'defacement','phishing','malware']))



      score = metrics.accuracy_score(y_test, y_pred_x)
      print("accuracy:   %0.3f" % score)
```

Fig. 4. XGBoost Classifier

### .2 Data Visualization

In this case study, we employed a Malicious URLs dataset **[8]** including 6,51,191 URLs, including 4,28,103 benign or safe URLs, 96,457 defacement URLs, 94,111 phishing URLs, and 32,520 malware URLs.

```
[2]: df =pd.read_csv('malicious_phish.csv')

[3]: df.head()

[3]:
```

| | url | type |
|---|---|---|
| 0 | br-icloud.com.br | phishing |
| 1 | mp3raid.com/music/krizz_kaliko.html | benign |
| 2 | bopsecrets.org/rexroth/cr/1.htm | benign |
| 3 | http://www.garage-pirenne.be/index.php?option=... | defacement |
| 4 | http://adventure-nicaragua.net/index.php?optio... | defacement |

Fig. 5. Dataset Specification

Figure 5 displays the dataset specifications and type, which includes two aspects: URL and label.

```
benign         428103
defacement      96457
phishing        94111
malware         32520
Name: type, dtype: int64
```

Fig 6. Analysis of Data

Figure 6 depicts a detailed study of data. The collection includes four sorts of URLs: benign (428,103), defacement (96,457), phishing (94,111), and malware (32,520). Python's Word Cloud module displays frequently occurring words in a given dataset.

### .2.1 Feature Extraction

Feature extraction refers to the process of representing or enhancing features to improve machine learning model performance. It will be easier for ML models to make better decisions. It reduces dimensionality, allowing for speedier processing. PCA and LDA are the most widely used approaches. Table I displays the retrieved characteristics.

TABLE 1. CHARACTERISTICS EXTRACTED FOR FEATURE ENGINEERING

| **Features** Use of IP | **Features** Google Index |
|---|---|
| Count(.) | Count(www) |
| Count(@) | Count(dir) |
| Count(embed_domain) | Count(http) |
| Count(%) | Count(?) |
| Count(-) | Count(=) |
| URL length | Hostname length |
| Count(Digits) | Count(letters) |

ML models cannot comprehend text directly, thus categorical information like IP address, URL, and Google Index are encoded as integers. Encoding is done using a count encoder. In the target column, benign websites are set to 0, whereas dangerous websites are assigned to 1. The distribution of categorical features in the dataset is shown.

Feature scaling allows for fixed-range scaling of data. This method is used to handle high variance data during pre-processing. Without data scaling, machine learning models tend to prioritize greater values over lower ones. Standardization is one of the two most common ways.

$$X^1 = \frac{X - \mu}{\sigma}$$

### .2.2 Normalization

This approach rescales values from 0 to one.Normalization removes duplicate data points (outliers) and organizes the data for greater efficiency.

$$X^1 = \frac{X - X_{\min}}{X_{\max} - X_{\min}}$$

### .2.3 Performance metrics

Performance metrics are distributed with an 80:20 ratio between training and testing. The evaluation metrics are:

$$\text{Accuracy} = \frac{(TP+TN)}{(TP+TN+FP+FN)}$$

$$\text{Precision} = \frac{(TP)}{(TP+FP)}$$

$$\text{Recall} = \frac{TP}{(TP+FP)}$$

$$F - Score = 2 * \frac{Precision * Recall}{Precision + Recall}$$

## Results ana Analysis

### .1 Simulation Results

The simulation was carried out using Google colab software. This web platform is open source and includes a variety of Python modules. It is user-friendly and adaptable.

| | precision | recall | f1-score | support |
|---|---|---|---|---|
| benign | 0.97 | 0.98 | 0.98 | 85621 |
| defacement | 0.98 | 0.99 | 0.99 | 19292 |
| phishing | 0.99 | 0.94 | 0.97 | 6504 |
| malware | 0.91 | 0.86 | 0.88 | 18822 |
| accuracy | | | 0.97 | 130239 |
| macro avg | 0.96 | 0.95 | 0.95 | 130239 |
| weighted avg | 0.97 | 0.97 | 0.97 | 130239 |

accuracy: 0.966

Fig. 7. Performance measure of Random Forest

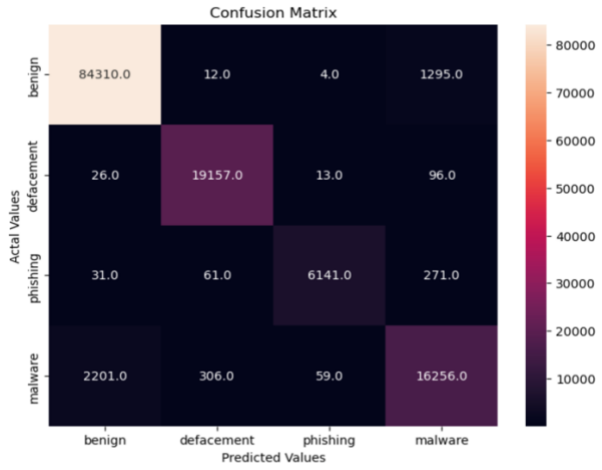



Fig. 8. Confusion matrix of Random Forest

Figures 7 and 8 display the performance measurements and confusion matrix for random forest classifiers. The results indicate that 84,334 benign, 19,154 defacement, 6,145 phishing, and 16,218 websites were accurately detected with 96.6% accuracy.

**Results of LightGBM Classifier**

| | precision | recall | f1-score | support |
|---|---|---|---|---|
| benign | 0.97 | 0.99 | 0.98 | 85621 |
| defacement | 0.96 | 0.99 | 0.98 | 19292 |
| phishing | 0.97 | 0.90 | 0.93 | 6504 |
| malware | 0.90 | 0.83 | 0.86 | 18822 |
| accuracy | | | 0.96 | 130239 |
| macro avg | 0.95 | 0.93 | 0.94 | 130239 |
| weighted avg | 0.96 | 0.96 | 0.96 | 130239 |

accuracy: 0.959

Fig 9. Performance Measure of LightGBM

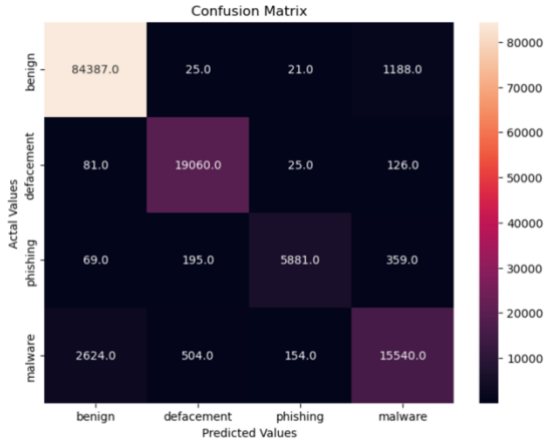



Fig 10. Confusion Matrix of LightGBM

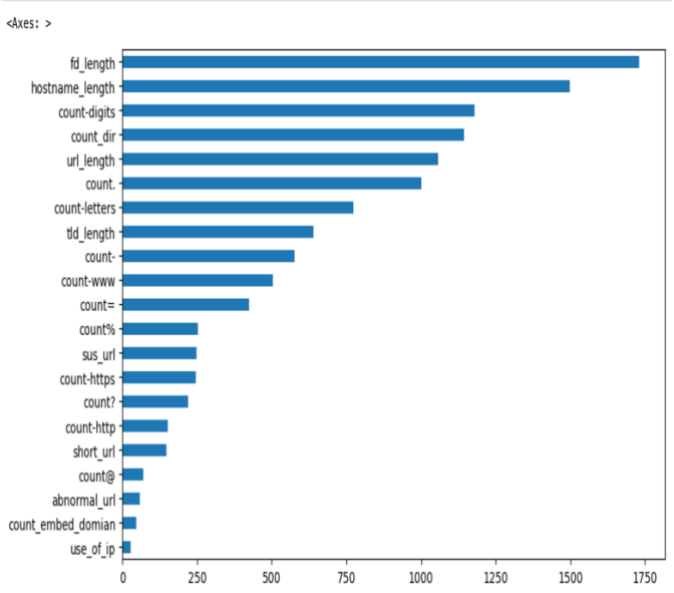



Fig 11. Feature Importance of LightGBM

Figures 9 and 10 display the performance measurements and confusion matrix for LightGBM classifiers. Figure 26 displays the confusion matrix for LightGBM classification, indicating the number of correctly predicted URLs for each dataset

category (Benign, Malware, Defacement, and Phishing). The results indicate that 84,389 benign, 19,018 defacement, 5,779 phishing, and 15,300 websites were accurately detected with 95.6% accuracy.

**Results of the XGBoost Classifier**

| | precision | recall | f1-score | support |
|---|---|---|---|---|
| benign | 0.97 | 0.99 | 0.98 | 85621 |
| defacement | 0.97 | 0.99 | 0.98 | 19292 |
| phishing | 0.98 | 0.91 | 0.94 | 6504 |
| malware | 0.91 | 0.83 | 0.87 | 18822 |
| accuracy | | | 0.96 | 130239 |
| macro avg | 0.96 | 0.93 | 0.94 | 130239 |
| weighted avg | 0.96 | 0.96 | 0.96 | 130239 |

accuracy: 0.962

Fig 12. Performance Measure of XGBoost

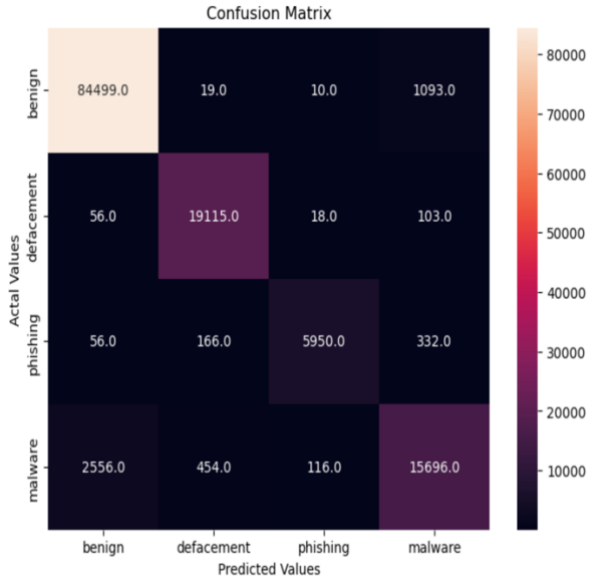



Fig 13. Confusion Matrix of XGBoost

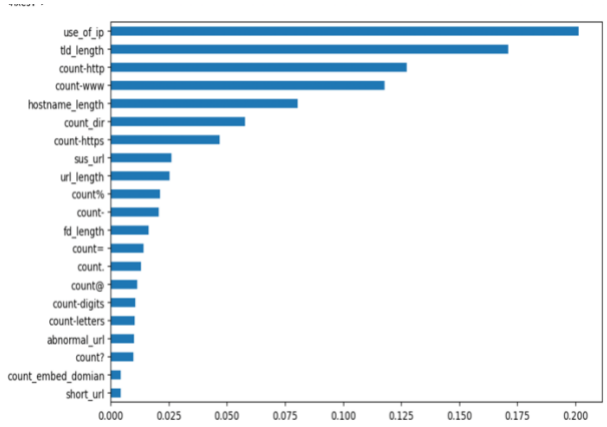



Fig 14. Feature Importance of XGBoost

Figures 12 and 13 display the performance measurements and confusion matrix for XGBoost classifiers. Results indicate a 93.2% accuracy rate in detecting 84,267 benign, 18,589 defacement, 4,912 phishing, and 13,655 websites.

## .2 Performance Analysis

TABLE II ANALYSIS OF PRECISION

| Algorithms/ Type | Benign | Defacement | Pishing | Malware |
|---|---|---|---|---|
| **Randome Forest** | 0.97 | 0.98 | 0.99 | 0.91 |
| **LightGBM** | 0.97 | 0.96 | 0.96 | 0.90 |
| **XGBoost** | 0.95 | 0.89 | 0.92 | 0.88 |

TABLE III ANALYSIS OF RECALL

| Algorithms/ Type | Benign | Defacement | Pishing | Malware |
|---|---|---|---|---|
| **Randome Forest** | 0.98 | 0.99 | 0.94 | 0.86 |
| **LightGBM** | 0.99 | 0.99 | 0.89 | 0.81 |
| **XGBoost** | 0.98 | 0.96 | 0.76 | 0.73 |

TABLE III ANALYSIS OF F1 SCORE

| Algorithms/ Type | Benign | Defacement | Pishing | Malware |
|---|---|---|---|---|
| **Randome Forest** | 0.98 | 0.99 | 0.96 | 0.88 |
| **LightGBM** | 0.98 | 0.97 | 0.92 | 0.85 |
| **XGBoost** | 0.97 | 0.92 | 0.83 | 0.80 |

Table II to IV compares precision, recall, and F1 scores for random forest, LightGBM, and XGBoost algorithms. A random forest classifier has the highest precision across all sorts of data. The LightGBM classifier achieves great classification performance by distributing the dataset leaf-wise. LightGBM Classifier provides the highest precision for benign URLs. The LightGBM classifier performed best for benign and defacement data, whereas the Random Forest Classifier excelled for defacement, phishing, and malware datasets. The random forest classifier achieves the highest F1 score across all types of datasets. LightGBM classifier achieves the highest F1 score for benign URLs.

## Refrences


1. P. Prakash, M. Kumar, R. R. Kompella, and M. Gupta, ‘‘PhishNet: Predictive blacklisting to detect phishing attacks,’’ in Proc. IEEE INFOCOM, San Diego, CA, USA, Mar. 2010, pp. 1–5.

2. Dharmaraj Rajaram Patil and JB Patil. 2015. Survey on Malicious Web Pages Detection Techniques. International Journal of u-and e-Service, Science and Technology (2015).

3. Schultz, G.M.; Eskin, E.; Zadok, F.; Stolfo, J.S. Data Mining Methods for Detection of New Malicious Executables. In Proceedings of the IEEE Computer Society Symposium on Research in Security and Privacy, Oakland, CA, USA, 13–16 May 2001; pp. 38–49.

4. Bai, J.; Wang, J.; Zou, G. A Malware Detection Scheme Based on Mining Format Information. Sci. World J. 2014, 2014, 260905. [CrossRef] [PubMed]

5. Moser, A.; Kruegel, C.; Kirda, E. Limits of Static Analysis for Malware Detection. In Proceedings of the Twenty-Third Annual Computer Security Applications Conference (ACSAC 2007), Miami Beach, FL, USA, 10–14 December 2007; pp. 421–430

6. Mohanta, A.; Saldanha, A. Malware Analysis and Detection Engineering: A Comprehensive Approach to Detect and Analyze Modern Malware, 1st ed.; Apress: New York, NY, USA, 2020; ISBN 978-148-426-192-7.

7. Rakesh Verma et al. "What's in a URL: Fast Feature Extraction and Malicious URL Detection", Seventh ACM Conference on Data and Application Security and Privacy, pp.55-63,2017.

8. https://www.kaggle.com/datasets/sid321axn/malicious-urls-dataset

9.

| Algorithms/ Type | Benign | Defacement | Pishing | Malware |
|---|---|---|---|---|
| **Randome Forest** | 0.98 | 0.99 | 0.94 | 0.86 |
| **LightGBM** | 0.99 | 0.99 | 0.89 | 0.81 |
| **XGBoost** | 0.98 | 0.96 | 0.76 | 0.73 |

TABLE III ANALYSIS OF RECALL